

EE-2003

Computer Organization and Assembly Language

Week 03 – Lecture 05-06

Department of Computer Science/Software Engineering/ Artificial Intelligence
FAST-NUCES Peshawar

September 1, 2025

Outline

- 1 Quick Review
- 2 Register Architecture
- 3 Data Declaration
- 4 Direct Addressing
- 5 Listing File Explanation
- 6 Example c02-02
- 7 Example c02-03
- 8 Example c02-04
- 9 Size Mismatch Errors-Example c02-05

Quick Review: What is Done Last Week?

- GPR Size and Machine Architecture
- Example Program in Assembly
- Endianness

INTEL iAPX 88 Architecture

- We use the **IBM PC based on Intel architecture** because of:
 - Wide availability
 - Free assemblers and debuggers
 - Applications in a variety of domains
- **iAPX 88** = “Intel Advanced Processor Extensions 88”
 - Also called the **8088**
 - Used in the first IBM PC machines
 - 16-bit processor (registers and accumulator are 16-bit)
 - Could address up to **1 MB memory** (vs 64 KB of 8085)
- **IBM PC XT** with 8088 became hugely successful:
 - Ran at **4.43 MHz** (fast at that time)
 - Open architecture made it popular worldwide
- Later: **iAPX 386** (32-bit) introduced
 - Downward compatible with 8088
 - All 8088 code runs on 386
 - More powerful and versatile

Register Architecture (iAPX 88)

- iAPX 88 has a total of **14 registers**
- Categories:
 - General Registers: AX, BX, CX, DX
 - Index Registers: SI, DI
 - Special Registers: IP, SP, BP
 - Flags Register
 - Segment Registers: CS, DS, SS, ES

Registers are the CPU's "working memory" – much faster than RAM.

General Registers

Register	Name / Purpose	Special Function
AX	Accumulator	Some instructions only work with AX
BX	Base	Used in memory addressing
CX	Counter	Loop counter in many instructions
DX	Destination	Destination in I/O operations

Table: General Purpose Registers (16-bit, split into High/Low parts)

- Example: **AX = AH (high byte) + AL (low byte)**
- Changes in AH or AL also reflect in AX.

Index Registers

- **SI (Source Index)** and **DI (Destination Index)**
- Hold memory addresses for data access.
- Frequently used in **string operations**.
- 16-bit registers (cannot be split into 8-bit parts).

Think of them as “pointers” for source and destination data.

Special Registers

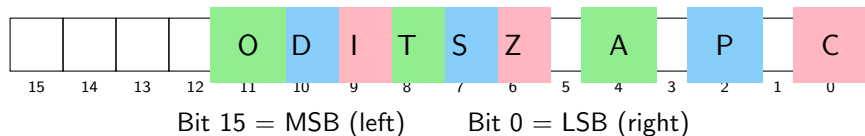
- **IP (Instruction Pointer)** – Holds address of next instruction
 - Controlled automatically by CPU
 - Dangerous to modify directly
- **SP (Stack Pointer)** – Points to the top of the stack
- **BP (Base Pointer)** – Used for stack addressing

Flags Register

Flag	Name	Meaning
C	Carry	Extra bit from addition/subtraction
P	Parity	Even/odd number of 1s
A	Aux. Carry	Carry from lower nibble (4 bits)
Z	Zero	Set if result is zero
S	Sign	Indicates positive/negative result
T	Trap	For debugging (step-by-step exec.)
I	Interrupt	Enable/disable external interrupts
D	Direction	String ops: forward/backward
O	Overflow	Wrong sign in signed arithmetic

Table: Important Flags in Intel Architecture

Flags Register (16-bit Layout)



Segment Registers

- Intel uses a **segmented memory model**.
- Registers:
 - **CS** – Code Segment
 - **DS** – Data Segment
 - **SS** – Stack Segment
 - **ES** – Extra Segment
- Each holds the starting address of a memory segment.
- Supports addressing more memory efficiently.

Data Declaration in Assembly

- First instruction: `mov ax, 5`
 - **Opcode:** MOV
 - **Destination:** AX
 - **Source:** 5 (immediate operand)
 - Example encoding: `B80500` → `B8` = opcode, `0500` = operand
- **Immediate operand** = constant embedded in instruction
- Constants cannot be modified; real programs need **variables in memory**
- Memory reserved using assembler directives:
 - `db` → define byte (8-bit)
 - `dw` → define word (16-bit)

Example:

```
num1    db    5        ; reserves 1 byte with value 5
num2    dw    1234      ; reserves 2 bytes with value 1234
```

Labels and Variables

- To use these variables, we need their **addresses**.
- The assembler associates a **symbol (label)** with an address.
- Labels act like variable names in high-level languages:
 - Programmer uses symbol (easy to read).
 - Assembler replaces it with the actual memory address.
- Example:
 - `num1: db 5` → assembler remembers address of `num1`.
 - Later instructions can use `num1` instead of raw address.

Global Variables and Register Size

- When we define variables at the top of the program, they are treated as **global variables**.

- Example:

```
num1    db    5        ; 8-bit variable
num2    dw    1234     ; 16-bit variable
```

- Issue: our registers (AX, BX, etc.) are **16-bit**, so:
 - Reading num2 (16-bit) fits naturally in registers.
 - Reading num1 (8-bit) requires extra care, since it occupies only part of the register.
- This mismatch between variable size and register size is a common source of confusion.
- Don't worry — this will become much clearer when we study **Examples c02-04 and c02-05**.

Direct Addressing Example

Program: Add three numbers using memory variables

```
[org 0x0100]
```

```
mov  ax, [num1]      ; load first number into AX
; mov [num1], [num2] ; illegal instruction
mov  bx, [num2]
add  ax, bx
mov  bx, [num3]
add  ax, bx
mov  [num4], ax
mov  ax, 0x4c00
int  0x21
num1: dw 5
num2: dw 10
num3: dw 15
num4: dw 0
```

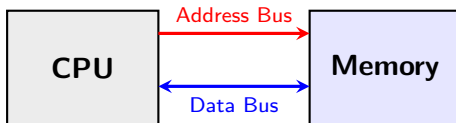
Understanding Direct Addressing

- `[num1]` means: take the value from the **memory location** labeled `num1`.
- Brackets `[ ]` signal that the operand is stored in memory, not an immediate constant.
- Example: `mov ax, [num1]` → load the contents of memory at `num1` into `AX`.
- The assembler replaces `num1`, `num2`, etc. with their actual memory addresses.
- `num1: dw 5` → assembler reserves 2 bytes in memory, initialized with value 5.
- Labels (like `num1`) are symbolic for humans, but become addresses for the processor.
- Illegal: `mov [num1], [num2]` — because both operands cannot be memory at the same time.

Remember!

- You have only **one address bus**.
- When moving data:
 - One operand must be in **memory**, the other a **register or immediate**.
 - Example: `mov ax, [num1]` ✓
- **No memory-to-memory movement!**
 - Example: `mov [num1], [num2]` ✗
 - CPU cannot place two memory addresses on one bus.
- Labels (`num1`, `num2`) are **addresses**, not values.

Remember! (Visualization)



Only one address bus → no memory-to-memory ops!

Listing File (Excerpt)

Assembler output (excerpt):

[org 0x0100]

```
00000000 A1[1700]    mov    ax, [num1]
00000003 8B1E[1900]  mov    bx, [num2]
00000007 01D8        add    ax, bx
00000009 8B1E[1B00]  mov    bx, [num3]
0000000D 01D8        add    ax, bx
0000000F A3[1D00]    mov    [num4], ax

00000012 B8004C      mov    ax, 0x4c00
00000015 CD21      int    0x21
00000017 0500 num1: dw    5
00000019 0A00 num2: dw    10
0000001B 0F00 num3: dw    15
0000001D 0000 num4: dw    0
```

Explanation (1/2)

- First instruction changed from B80500 to A11700.
- B8 = move constant into AX.
- A1 = move from memory into AX.
- Immediate operand 1700 = offset of num1.
- Assembler replaced num1 with its real address (0017).

Explanation (2/2)

- num2, num3, and num4 at 0019, 001B, 001D.
- In debugger at offset 0100 → instruction becomes `mov ax, [0117]`.
- Data window confirms offset 0117 contains 0005.
- Debugger shows data as “fake code” → terminate execution before it reaches data.

Observations

- Labels (`num1`, `num2`) = symbols for us; assembler converts them to numbers.
- Debugger only shows numeric addresses (0117, 0119, ...).
- `dw` stores two-byte words (0005).
- `db` would store only one byte (05).
- We can also use expressions like `num1+2` instead of `num2`.

Example c02-02: Add Three Numbers (Single Label Reference)

Assembly source:

```
[org 0x0100]
```

```
mov  ax, [num1]           ; load first number in ax
mov  bx, [num1+2]         ; load second number in bx
add  ax, bx               ; accumulate sum in ax
mov  bx, [num1+4]         ; load third number in bx
add  ax, bx               ; accumulate sum in ax
mov  [num1+6], ax         ; store sum at num1+6
mov  ax, 0x4c00           ; terminate program
int  0x21

num1: dw 5
      dw 10
      dw 15
      dw 0
```

Explanation of c02-02

- num2, num3, and num4 are removed.
- All data is accessed with reference to num1.
- For example:
 - Second number $\rightarrow$ num1+2
 - Third number $\rightarrow$ num1+4
 - Result $\rightarrow$ num1+6
- Expression num1+2 is evaluated at assembly time $\rightarrow$ assembler replaces it with actual address (e.g., 0117+2=0119).
- When opened in the debugger, the program looks identical to the previous version because assembler handled the offset math.

Example c02-03: Add Three Numbers (Compact Declaration)

Assembly source:

```
[org 0x0100]
```

```
mov  ax, [num1]          ; load first number in ax
mov  bx, [num1+2]        ; load second number in bx
add  ax, bx              ; accumulate sum in ax
mov  bx, [num1+4]        ; load third number in bx
add  ax, bx              ; accumulate sum in ax
mov  [num1+6], ax        ; store sum at num1+6
```

```
mov  ax, 0x4c00          ; terminate program
int  0x21
```

```
num1: dw 5, 10, 15, 0
```

Explanation of c02-03

- Data is declared in a **single line**: `dw 5, 10, 15, 0`.
- Saves space and avoids the need for multiple labels (`num2`, `num3`, etc.).
- This approach still places four words in **consecutive memory locations**.
- The address of the first word is labeled `num1`; subsequent data is accessed using offsets (`num1+2`, `num1+4`, etc.).
- This method demonstrates **direct addressing**:
 - Memory addresses are fixed in the instruction.
 - Operands must be the same size (word $\leftrightarrow$ word register).
- Note: Value stored in memory 0500 is read into the register as 0005 — registers maintain correct word ordering.

Example c02-04: Add Three Numbers (Directly in Memory)

Assembly source:

```
[org 0x0100]
```

```
mov ax, [num1]
mov [num1 + 6], ax      ; add this value to result
```

```
mov ax, [num1 + 2]
add [num1 + 6], ax
```

```
mov ax, [num1 + 4]
add [num1+6], ax
```

```
mov ax, 0x4c00
```

```
int 0x21
```

```
num1:  dw  5, 10, 15, 0
```

Listing File for Example c02-04

```
[org 0x0100]
00000000 A1[1900]      mov  ax, [num1]
00000003 A3[1F00]      mov  [num1+6], ax
00000006 A1[1B00]      mov  ax, [num1+2]
00000009 0106[1F00]    add  [num1+6], ax
0000000D A1[1D00]      mov  ax, [num1+4]
00000010 0106[1F00]    add  [num1+6], ax
00000014 B8004C      mov  ax, 0x4c00
00000017 CD21        int  0x21
00000019 05000A000F0000 num1: dw 5, 10, 15, 0
```

Explanation of Example c02-04

- The opcode of add changes because the **destination is memory** instead of a register.
- Some opcodes are longer as a result of using memory operands.
- num1 is automatically translated by the assembler into the correct memory offset.
- Method of access: still **direct addressing**, whether with MOV or ADD.
- During execution:
 - Word data is read into registers in the correct order.
 - Destination of addition is now memory instead of AX.

Why Memory-to-Memory Operations Are Illegal

Illegal attempt:

```
mov [num1+6], [num1]    ; ILLEGAL
```

- Intel architecture does **not allow memory-to-memory transfers**.
- Only one operand can be a memory address in most instructions.
- Allowed operations:
 - Register $\leftrightarrow$ Register
 - Register $\leftrightarrow$ Memory
 - Constant $\rightarrow$ Register / Memory
- Disallowed:
 - Memory $\leftrightarrow$ Memory
 - Register $\rightarrow$ Constant
 - Memory $\rightarrow$ Constant
- Only **string instructions** allow memory-to-memory movement (covered later).

Size Mismatch Errors-Example c02-05

- If we declare variables with `dw` (word) but access them as bytes → results are incorrect.
- Example: changing `dw` to `db` still assembles, but the execution gives **unexpected results**.
- Why? Because data alignment and offsets no longer match.
- This is a **logic error**, not a syntax error.
- The assembler only prevents **illegal instructions**, not logical mistakes.

Example c02-05: Add Three Numbers Using Bytes

Assembly source:

```
[org 0x0100]

mov  al, [num1]      ; load first number in AL
mov  bl, [num1+1]    ; load second number in BL
add  al, bl          ; accumulate sum in AL
mov  bl, [num1+2]    ; load third number in BL
add  al, bl          ; accumulate sum in AL
mov  [num1+3], al     ; store sum at num1+3

mov  ax, 0x4c00       ; terminate program
int  0x21

num1: db  5, 10, 15, 0
```

Explanation of c02-05

- Using `db` → each variable takes 1 byte.
- Offsets move in **bytes**, not words:
 - `num1` at offset `+0`
 - `num1+1` → 2nd number
 - `num1+2` → 3rd number
 - `num1+3` → result
- Correct registers are used: `AL/BL` (8-bit) match `db`.
- No alignment or synchronization issues this time.
- Lesson: **Data size in declaration must match register size in access.**

Important Rules for Size Matching

- Register size $\leftrightarrow$ Memory size must always match.
- Examples:
 - `mov ax, [num1]` $\rightarrow$ word access (16-bit).
 - `mov al, [num1]` $\rightarrow$ byte access (8-bit).
- Illegal example: `mov ax, bl` (word vs byte).
- Ambiguous example: `mov [num1], 5`
 - Processor cannot guess if 5 is byte or word.
 - Must specify explicitly:
 - `mov byte [num1], 5`
 - `mov word [num1], 5`
- Assembly gives you power $\rightarrow$ but you are responsible!

DB vs DW: Memory Layout

Why offsets change with `db` and `dw`:

Using `dw` (word = 2 bytes):

Address	Data
0	05 00
2	0A 00
4	0F 00
6	00 00

Second number is at `num1+2` → word offset.

Using `db` (byte = 1 byte):

Address	Data
0	05
1	0A
2	0F
3	00

Second number is at `num1+1` → byte offset.

Key Point:

- With `dw`, each number takes **2 bytes**, so offsets jump by 2.
- With `db`, each number takes **1 byte**, so offsets jump by 1.

Thank You!

Any Questions?

- Today we explored: **data declarations, direct addressing, size mismatches.**
- Key lesson: **Programmer is responsible for correct size and access in assembly.**
- Assembly is powerful but requires careful and precise handling.

“If you understand the CPU, you understand the computer.”